

Resolução do Exame Modelo de Recurso — Bases de Dados (Modelo 4)

 **Ano Letivo:** 2025/2026 |  **Época:** Recurso (Modelo)

 **Curso:** Engenharia Informática / Segurança Informática em Redes de Computadores

 **Instituição:** P.PORTO — ESTG

 **Unidade Curricular:** Bases de Dados

1. Componentes do Ambiente de um SGBD (2 val.)

 **Pergunta 1:** Descreva os cinco componentes principais do ambiente de um Sistema de Gestão de Bases de Dados (SGBD) e explique sumariamente como eles se relacionam entre si.

Resposta:

Os cinco componentes principais do ambiente de um SGBD são:

1. **Hardware:** A parte física do sistema, composta pelos servidores, processadores (CPUs), memória RAM, dispositivos de armazenamento persistente (discos rígidos/SSDs) e infraestrutura de rede necessária para correr o software e armazenar os dados.
2. **Software:** O motor do SGBD propriamente dito, o sistema operativo do servidor, os softwares de rede e as aplicações cliente que acedem à base de dados.
3. **Dados:** O elemento central do sistema. Inclui os dados operacionais (registos inseridos pelas aplicações) e os metadados (a descrição da estrutura, tabelas e regras de integridade guardadas no dicionário de dados).
4. **Utilizadores:** As pessoas que interagem com o sistema, classificadas em: administradores de bases de dados (DBAs, que gerem e mantêm o sistema), programadores (que desenvolvem aplicações que comunicam com a BD) e utilizadores finais (que consultam ou atualizam dados).
5. **Procedimentos:** As regras, diretrizes, políticas de segurança e instruções que orientam a utilização e a manutenção do sistema (como políticas de backups, recuperação de desastres, criação de contas e manutenção de índices).

Relacionamento entre componentes:

O **Hardware** fornece a infraestrutura computacional onde o **Software** (o SGBD) corre. O SGBD, por sua vez, processa e gere os **Dados** físicos e lógicos, segundo as políticas e rotinas definidas nos **Procedimentos** operacionais. Tudo isto serve para responder às necessidades e comandos dos **Utilizadores**, que interagem com o software para ler ou alterar a informação.

2. 🗝️ Conceitos do Modelo Relacional (2 val.)

? **Pergunta 2:** No contexto do modelo relacional de bases de dados, explique detalhadamente o significado de cada um dos seguintes termos: Relação, Atributo, Domínio, Tuplo, Grau e Cardinalidade.

👉 Resposta:

- **Relação:** No modelo relacional, uma relação corresponde a uma tabela de dados com um nome único. É uma estrutura bidimensional composta por colunas (atributos) e linhas (tuplos) que partilham uma semântica comum.
- **Atributo:** Representa uma coluna da tabela (relação). Corresponde a uma propriedade ou característica específica que descreve a entidade representada (ex.: o atributo `NIF` ou `Nome` na tabela `Cliente`).
- **Domínio:** Representa o conjunto de valores válidos e admissíveis que um determinado atributo pode assumir. O domínio define o tipo de dados (ex.: inteiros, texto, datas) e possíveis restrições de gama (ex.: valores maiores que zero).
- **Tuplo:** Representa uma linha da tabela (relação). Corresponde a uma ocorrência concreta de registo de dados (uma instância) da relação, contendo um valor específico para cada atributo.
- **Grau:** É o número total de atributos (colunas) que constituem a relação. Por exemplo, uma tabela com as colunas `codCliente`, `nome` e `nif` tem grau 3.
- **Cardinalidade:** É o número total de tuplos (linhas) atualmente existentes e armazenados na tabela. Ao contrário do grau, que é relativamente fixo, a cardinalidade é dinâmica e varia à medida que são inseridos ou removidos registos.

3. Operações de Junção (2 val.)

? Pergunta 3: Descreva as diferenças existentes entre as seguintes cinco operações de junção no modelo relacional, indicando o operador lógico/comparação utilizado e se a operação resulta na duplicação ou eliminação de colunas equivalentes: Theta Join, Equijoin, Natural Join, Outer Join e Semijoin.

Resposta:

- **Theta Join (θ -Join):** É a forma mais genérica de junção. Combina registos de duas tabelas com base em qualquer operador de comparação lógico ou aritmético ($=$, $>$, $<$, $\geq$, $\leq$, $\neq$). O resultado mantém todas as colunas de ambas as tabelas (sem eliminação de duplicados).
- **Equijoin:** É um caso particular do Theta Join onde a condição de junção utiliza exclusivamente o operador de igualdade ($=$) entre as colunas relacionadas. Tal como no Theta Join, mantém as colunas comparadas duplicadas no resultado (ex.: terá a coluna `codCliente` de ambas as tabelas lado a lado).
- **Natural Join (Junção Natural):** É uma junção de igualdade ($=$) que atua de forma automática sobre as colunas que possuem exatamente o mesmo nome em ambas as tabelas. Ao contrário do Equijoin, o Natural Join **elimina as colunas duplicadas** no resultado final, projetando apenas uma coluna para cada par de atributos idênticos.
- **Outer Join (Junção Externa):** Combina os dados baseando-se na igualdade ($=$), mas inclui também os registos que não encontram correspondência na outra tabela. Divide-se em:
 - *Left Outer Join:* Preserva todas as linhas da tabela à esquerda.
 - *Right Outer Join:* Preserva todas as linhas da tabela à direita.
 - *Full Outer Join:* Preserva todas as linhas de ambas as tabelas.
Preenche com valores `NULL` os atributos da tabela onde não houve correspondência. Não remove colunas duplicadas.
- **Semijoin (Junção Parcial):** Retorna exclusivamente as linhas da primeira tabela que possuem correspondência na segunda tabela. Ao contrário das restantes junções, **não combina as colunas** das duas tabelas: o resultado final contém unicamente os atributos da primeira tabela, eliminando qualquer duplicação proveniente do cruzamento de dados.

4. 📁 Procedimentos vs Funções (2 val.)

? **Pergunta 4:** Qual a diferença entre um Procedimento Armazenado (*Stored Procedure*) e uma Função Definida pelo Utilizador (*User-Defined Function*) numa base de dados relacional? Aponte três diferenças fundamentais e indique em que situações é preferível cada um.

👉 Resposta:

As três diferenças fundamentais entre Procedimentos Armazenados e Funções são:

1. Obrigação e Estrutura de Retorno:

- *Função (UDF)*: Tem a obrigatoriedade de retornar sempre um valor (um escalar como `int` ou `varchar`, ou um conjunto tabular de dados).
- *Procedimento (SP)*: Não tem a obrigatoriedade de retornar valores (pode retornar conjuntos de resultados, parâmetros de saída `OUTPUT`, códigos de estado ou simplesmente nada).

2. Contexto de Execução / Chamada:

- *Função (UDF)*: Pode ser incorporada diretamente como parte de uma expressão SQL, como nas cláusulas `SELECT`, `WHERE` ou `JOIN` (ex: `SELECT nome, dbo.CalcularDesconto(preco) FROM Produtos`).
- *Procedimento (SP)*: Não pode ser chamado dentro de queries SQL. Deve ser executado isoladamente por meio do comando `CALL` ou `EXEC` (ex: `EXEC RegistrarVenda @id=10`).

3. Efeitos Secundários (Side Effects) e Modificação de Dados:

- *Função (UDF)*: É estritamente *read-only* em relação ao estado físico da BD. Não pode executar operações DML (`INSERT`, `UPDATE`, `DELETE`) em tabelas permanentes, nem transações.
- *Procedimento (SP)*: Pode efetuar qualquer operação DML ou DDL, controlar e gerir transações (`COMMIT`, `ROLLBACK`) e modificar livremente os dados físicos.

Situações de Preferência:

- **Usar Função (UDF) quando:** O objetivo é efetuar um cálculo matemático complexo, formatar texto, validar um valor ou construir uma tabela derivada dinâmica que necessite de ser consultada diretamente dentro de uma instrução `SELECT`.
- **Usar Procedimento (SP) quando:** O objetivo é executar processos ou lógica de negócio complexa que envolva escrita na base de dados (inserções, atualizações em lote), controlo transacional completo ou tarefas administrativas recorrentes.

5. Vistas Atualizáveis (2 val.)

? Pergunta 5: Quais as restrições e condições necessárias para garantir que uma vista (*view*) tradicional seja atualizável diretamente através de instruções DML (como INSERT, UPDATE ou DELETE) sobre as tabelas base sem recorrer a triggers?

Resposta:

Para que uma vista seja atualizável diretamente através de instruções DML, as alterações feitas na vista devem poder ser mapeadas sem ambiguidade para uma única tabela física subjacente. Para tal, a vista deve obedecer às seguintes restrições:

- 1. Focar-se numa Única Tabela Base:** Qualquer instrução de escrita (`INSERT` , `UPDATE` , `DELETE`) apenas pode afetar uma tabela física de cada vez. Vistas baseadas em junções (`JOINS`) de múltiplas tabelas têm severas restrições (ex: no `UPDATE` só se podem alterar colunas de uma das tabelas; `INSERT` ou `DELETE` são geralmente rejeitados pelo motor).
- 2. Conter as Colunas Obrigatórias:** Para efetuar `INSERT` pela vista, esta deve expor a Chave Primária e todas as colunas que tenham a restrição `NOT NULL` (e que não possuam um valor por defeito/ `DEFAULT` ou propriedade de autoincremento/ `IDENTITY`).
- 3. Ausência de Funções de Agregação:** A vista não pode conter no seu `SELECT` funções como `SUM()` , `AVG()` , `COUNT()` , `MAX()` ou `MIN()` , pois estes dados são derivados e não mapeáveis diretamente para linhas físicas.
- 4. Ausência de Cláusulas de Agrupamento e Filtragem de Grupos:** A query da vista não pode conter as cláusulas `GROUP BY` ou `HAVING` .
- 5. Não Utilizar Operações de Conjuntos ou Exclusão:** A vista não pode conter as instruções `DISTINCT` ou `UNION / UNION ALL` .
- 6. Ausência de Atributos Derivados na Escrita:** Não se podem atualizar colunas calculadas na vista (ex: `salario * 1.10 AS NovoSalario`).

6. 🎨 Especialização vs Generalização no Modelo ER (2 val.)

? **Pergunta 6:** Explique as diferenças entre o processo de especialização e de generalização no contexto da modulação de dados com o diagrama Entidade-Relacionamento (ER), fornecendo exemplos práticos para cada um.

👉 Resposta:

Tanto a especialização como a generalização servem para criar hierarquias de superentidades e subentidades no modelo ER, mas representam abordagens em sentidos opostos:

- **Especialização (Abordagem Top-Down / Cima para Baixo):**
 - *Conceito:* É o processo de dividir uma entidade genérica em subentidades mais específicas com base em características distintivas. Identificam-se subconjuntos de ocorrências que possuem atributos ou relacionamentos próprios exclusivos.
 - *Exemplo prático:* A superentidade `Funcionario` pode ser especializada nas subentidades `Engenheiro` (com o atributo exclusivo `carteiraProfissional`), `Secretario` (atributo `velocidadeDigitacao`) e `Motorista` (atributo `categoriaCarta`).
- **Generalização (Abordagem Bottom-Up / Baixo para Cima):**
 - *Conceito:* É o processo inverso da especialização. Consiste em identificar atributos e comportamentos comuns a duas ou mais entidades distintas e agrupá-las numa única superentidade genérica, evitando a duplicação de dados no modelo.
 - *Exemplo prático:* As entidades individuais `Carro` e `Camiao` possuem atributos semelhantes (como `Matricula`, `Marca`, `AnoFabrico`). Podem ser generalizadas numa superentidade comum chamada `Veiculo`, mantendo apenas atributos exclusivos (como `numLugares` no carro, ou `capacidadeCarga` no camião) nas respetivas subentidades.

7. Exercício de Normalização de Fatura (3 val.)

? **Pergunta 7:** Normalização do recibo da Clínica Geral do Norte.

 **Resposta:**

Identificação dos Atributos

Letra	Atributo
A	NIF_Clinica
B	Nome_Clinica
C	Morada_Clinica
D	CodPostal_Clinica
E	NumRecibo
F	DataEmissao
G	HoraEmissao
H	ATCUD
I	NumUtenteSNS
J	NIF_Paciente
K	Nome_Paciente
L	Seguradora
M	CodPlano
N	RefServico
O	DescricaoServico
P	Prestador
Q	CodPrestador
R	PrecoBruto
S	CopagSeguradora

Letra	Atributo
T	UtenteParte
U	TaxaIVA_Linha
V	SubtotalLinha
W	TaxaIVA_Resumo
X	Incidencia_IVA
Y	Valor_IVA
Z	TotalSeguradora
AA	TotalUtente
AB	TotalGeral
AC	MetodoPagamento

Forma Não Normalizada (FNN)

```
Recibo_FNN(E, A, B, C, D, F, G, H, I, J, K, L, M, Z, AA, AB, AC,  
           [Q, N, O, P, R, S, T, U, V],  
           [W, X, Y])
```


1 Primeira Forma Normal (1FN)

Definição: Atributos devem possuir valores atômicos (sem grupos repetidos).

```
Recibo_1FN(NumRecibo, RefServico, TaxaIVA_Resumo,  
           NIF_Clinica, Nome_Clinica, Morada_Clinica, CodPostal_Clinica,  
           DataEmissao, HoraEmissao, ATCUD, NumUtenteSNS, NIF_Paciente,  
           Nome_Paciente, Seguradora, CodPlano, TotalSeguradora, TotalUtente,  
           TotalGeral, MetodoPagamento, DescricaoServico, Prestador, CodPrestador,  
           PrecoBruto, CopagSeguradora, UtenteParte, TaxaIVA_Linha, SubtotalLinha,  
           Incidencia_IVA, Valor_IVA)
```

PK: (NumRecibo, RefServico, TaxaIVA_Resumo)

Dependências Funcionais (DF) identificadas:

- \$NumRecibo \rightarrow NIF_Clinica, Nome_Clinica, Morada_Clinica, CodPostal_Clinica, DataEmissao, HoraEmissao, ATCUD, NumUtenteSNS, NIF_Paciente, Nome_Paciente, Seguradora, CodPlano, TotalSeguradora, TotalUtente, TotalGeral, MetodoPagamento\$
- \$NumRecibo, RefServico \rightarrow CopagSeguradora, UtenteParte, SubtotalLinha\$
- \$RefServico \rightarrow DescricaoServico, Prestador, CodPrestador, PrecoBruto, TaxaIVA_Linha\$
- \$NumRecibo, TaxaIVA_Resumo \rightarrow Incidencia_IVA, Valor_IVA\$
- \$NIF_Clinica \rightarrow Nome_Clinica, Morada_Clinica, CodPostal_Clinica\$
- \$NIF_Paciente \rightarrow Nome_Paciente, NumUtenteSNS\$
- \$CodPlano \rightarrow Seguradora\$

2 Segunda Forma Normal (2FN)

Definição: Eliminação de dependências parciais sobre a chave primária composta.

```
Cabecalho_2FN(NumRecibo, NIF_Clinica, Nome_Clinica, Morada_Clinica, CodPostal_Clinica,  
              DataEmissao, HoraEmissao, ATCUD, NumUtenteSNS, NIF_Paciente,  
              Nome_Paciente, Seguradora, CodPlano, TotalSeguradora, TotalUtente,  
              TotalGeral, MetodoPagamento)
```

PK: NumRecibo

```
Servico_2FN(RefServico, DescricaoServico, Prestador, CodPrestador, PrecoBruto, TaxaIVA_Lin  
ha)
```

PK: RefServico

```
LinhaRecibo_2FN(NumRecibo, RefServico, CopagSeguradora, UtenteParte, SubtotalLinha)
```

PK: (NumRecibo, RefServico)

```
ResumoIVA_2FN(NumRecibo, TaxaIVA_Resumo, Incidencia_IVA, Valor_IVA)
```

PK: (NumRecibo, TaxaIVA_Resumo)

3 Terceira Forma Normal (3FN)

Definição: Eliminação de dependências transitivas.

Na tabela `Cabecalho_2FN` verificamos as seguintes dependências transitivas:

- $\$NIF_Clinica \rightarrow Nome_Clinica, Morada_Clinica, CodPostal_Clinica\$$ (via NumRecibo)
- $\$NIF_Paciente \rightarrow Nome_Paciente, NumUtenteSNS\$$ (via NumRecibo)
- $\$CodPlano \rightarrow Seguradora\$$ (via NumRecibo)

Extraíndo estas dependências, obtemos o **Esquema Relacional Final (3FN)**:

```
Clinica(NIF_Clinica, Nome_Clinica, Morada_Clinica, CodPostal_Clinica)
    PK: NIF_Clinica

Paciente(NIF_Paciente, Nome_Paciente, NumUtenteSNS)
    PK: NIF_Paciente

PlanoSaude(CodPlano, Seguradora)
    PK: CodPlano

Servico(RefServico, DescricaoServico, Prestador, CodPrestador, PrecoBruto, TaxaIVA)
    PK: RefServico

Recibo(NumRecibo, DataEmissao, HoraEmissao, ATCUD, NIF_Clinica, NIF_Paciente, CodPlano,
    TotalSeguradora, TotalUtente, TotalGeral, MetodoPagamento)
    PK: NumRecibo
    FK: NIF_Clinica → Clinica(NIF_Clinica)
    FK: NIF_Paciente → Paciente(NIF_Paciente)
    FK: CodPlano → PlanoSaude(CodPlano)

LinhaRecibo(NumRecibo, RefServico, CopagSeguradora, UtenteParte, SubtotalLinha)
    PK: (NumRecibo, RefServico)
    FK: NumRecibo → Recibo(NumRecibo)
    FK: RefServico → Servico(RefServico)

ResumoIVA(NumRecibo, TaxaIVA, Incidencia_IVA, Valor_IVA)
    PK: (NumRecibo, TaxaIVA)
    FK: NumRecibo → Recibo(NumRecibo)
```

8. Modelação, SQL e Álgebra Relacional (5 val.)

a) Chaves primárias e estrangeiras da tabela PecaUtilizada (1 val.)

- **Chave Primária (PK):** (codOrdem, codPeca) .
 - *Justificação:* A tabela representa uma relação N:M (muitos-para-muitos) entre as entidades `OrdemReparacao` e `Peca` . A chave primária deve ser composta por ambas as chaves estrangeiras para garantir que uma mesma peça não seja registada múltiplas vezes na mesma ordem de reparação de forma duplicada, mantendo a unicidade da linha.
- **Chaves Estrangeiras (FK):**
 - `codOrdem` que referencia `OrdemReparacao(codOrdem)` .
 - `codPeca` que referencia `Peca(codPeca)` .
 - *Justificação:* Asseguram a integridade referencial: não é possível associar consumos a reparações ou peças que não existam previamente no sistema.

b) SQL (2 val.)

Pretende-se identificar peças 'Ecrãs' usadas em mais de 10 reparações concluídas no 1º Semestre de 2026, com preço debitado superior ao preço padrão.

```
SELECT P.codPeca, P.descricao
FROM Peca P
INNER JOIN PecaUtilizada PU ON P.codPeca = PU.codPeca
INNER JOIN OrdemReparacao O ON PU.codOrdem = O.codOrdem
WHERE P.categoria = 'Ecrãs'
      AND O.estado = 'Concluído'
      AND O.dataConclusao >= '2026-01-01'
      AND O.dataConclusao <= '2026-06-30'
      AND PU.precoDebitado > P.precoVenda
GROUP BY P.codPeca, P.descricao
HAVING COUNT(DISTINCT O.codOrdem) > 10;
```

c) Álgebra Relacional (2 val.)

Pretende-se identificar os dispositivos que foram reparados (estado = 'Concluído') mas nos quais **nunca** se utilizou nenhuma peça da categoria 'Baterias'.

Utilizamos o padrão de negação (\$T - A\$):

$\text{\$TodosReparados} \leftarrow \pi_{\{\text{numSerie}, \text{marca}, \text{modelo}\}}(\text{Dispositivo} \bowtie \sigma_{\{\text{estado} = \text{'Concluído'}\}}(\text{OrdemReparacao}))$

$\text{\$Baterias} \leftarrow \sigma_{\{\text{categoria} = \text{'Baterias'}\}}(\text{Peca})$

$\text{\$PecasUsadasBateria} \leftarrow \text{PecaUtilizada} \bowtie \text{Baterias}$

$\text{\$OrdensBateria} \leftarrow \pi_{\{\text{codOrdem}\}}(\text{PecasUsadasBateria})$

$\text{\$ReparacoesBateria} \leftarrow \text{OrdemReparacao} \bowtie \text{OrdensBateria}$

$\text{\$SeriesComBateria} \leftarrow \pi_{\{\text{numSerie}\}}(\text{ReparacoesBateria})$

$\text{\$SeriesSemBateria} \leftarrow \pi_{\{\text{numSerie}\}}(\text{TodosReparados}) - \text{SeriesComBateria}$

$\text{\$Resultado} \leftarrow \text{TodosReparados} \bowtie \text{SeriesSemBateria}$